

Dynamic Programming for Prediction Markets

Princeton Liu
206267879

I. INTRODUCTION

Kalshi's KXBTC15M market is a sequence of binary contracts, each asking whether Bitcoin will close higher than its current price fifteen minutes from now. A YES contract pays \$1 if it does and \$0 if it does not, and in between it trades at a price in $[0, 1]$ that the market continuously updates. This makes it a good fit for finite-horizon dynamic programming. The horizon is short and known ($T = 15$ one-minute steps), the terminal payoff is binary and objective, the public state (contract quotes, the underlying Bitcoin price, and the stream of executed trades) is fully observable, and thousands of independent episodes accumulate every month. Real decision problems with financial stakes rarely have a structure this clean for exact backward induction.

We study whether a dynamic programming or reinforcement learning agent can trade this market profitably. Our empirical results largely indicate that it cannot, suggesting that the observed limitations arise from the methodology rather than from insufficient model capacity.

The first of our three contributions is a verified exact-DP implementation on real market data, for which we build a discretized optimal-trading MDP from 6,331 real KXBTC15M episodes, solve it exactly by backward induction, and check the solver against small instances we can work out by hand. That check caught a bug in the value recursion, and the corrected results supersede our Week 6-7 milestone numbers (Section V). The second is a comparison between model-based and model-free approaches. Alongside the tabular DP, we train a Double DQN agent directly on the continuous state space, allowing us to separate errors caused by transition-model estimation from limitations of the trading signal itself. The third is a study of execution assumptions and model error. We run the same optimal policy under three execution models, where a taker crosses the spread, an optimistic maker assumes its resting orders always fill, and a pessimistic maker assumes they fill only when the market trades through them. The optimistic assumption is the default in most backtests, and we show that it creates value where none exists. Real maker performance falls somewhere between the optimistic and pessimistic cases, and on held-out data the whole band is negative. We account for the gap between positive in-sample value and negative out-of-sample performance with two independent tools. The first is a no-arbitrage check that certifies a lower bound on the transition model's value error, without our needing to know the true dynamics. The second is a simulation-lemma bound, evaluated with the transition errors we measure directly. Both

point to the same conclusion, that the gap is a discretization artifact, not a sign of market inefficiency.

II. RELATED WORKS

Sequential trading under uncertainty has long been studied as a stochastic control problem. The Almgren-Chriss model of optimal execution [1] treats trading as a finite-horizon control problem that trades off impact cost (the price moved against oneself by trading too quickly) against price risk (the price drifting while one waits). This is the starting point for our own discrete-time formulation, in which the state tracks price, position, and time. Prediction markets, in turn, are usually studied as probability aggregators. Wolfers and Zitzewitz [2] survey their efficiency properties, and Snowberg and Wolfers [3] document the favorite-longshot bias, a persistent miscalibration at extreme prices. That bias matters directly for us, since our contracts often trade near \$0 or \$1. More recent work studies Kalshi's crypto-linked contracts directly and asks whether their prices carry volatility information about the underlying [4].

On the methods side, model-free value learning goes back to Q-learning [5]. Fischer [6] surveys reinforcement learning in financial markets and notes that results are often highly sensitive to the assumed transaction costs and execution model. We put that sensitivity at the center of our study. Our learned-agent baselines use potential-based reward shaping [7], and our analysis uses the simulation lemma of Kearns and Singh [8] to link transition-model error to value error. We do not propose a new algorithm. What we add to this literature is an exact, verified DP treatment of a real prediction market, one where the execution model and the discretization, not the learning method, turn out to determine the conclusion.

III. PROBLEM FORMULATION

Each 15-minute market is one episode of $T = 15$ one-minute steps. At step t the public data give the YES contract's best bid b_t (the highest price a buyer is currently willing to pay) and best ask a_t (the lowest price a seller is willing to accept), both in dollars with $0 \leq b_t \leq a_t \leq 1$, together with features of the underlying Bitcoin price. The market settles at $p_T = \mathbf{1}\{\text{YES}\}$, where YES means the settlement value (per the market rules, the average of 60 CF Benchmarks Real-Time Index prices over the final minute) exceeds the strike, the reference price fixed when the window opens.

We discretize the market to a tabular state $x = (t, b, c)$ comprising the time step, the decile bin $b \in \{0, \dots, 9\}$ of the Kalshi mid-price $p_t = (b_t + a_t)/2$, and the decile bin $c \in \{0, \dots, 9\}$ of the cumulative Bitcoin log-return since the

window opened (deciles computed on training data). A held position additionally remembers its entry price bin e .

When holding no position (flat), the agent chooses among NEUTRAL, LONG (buy YES), and SHORT (buy NO, which is the same as selling YES). Entries are allowed only for $t_{\min} \leq t \leq T - 2$ with $t_{\min} = 3$. Once in a position, the agent chooses HOLD or EXIT. Any position still open at T settles at the outcome, fee-free.

The pair (b, c) evolves under a factored, time-inhomogeneous kernel

$$\hat{P}(b', c' | b, c, t) = \hat{P}_1(b' | b, t) \hat{P}_2(c' | c, t), \quad (1)$$

estimated from training episodes (Section IV). Factoring assumes the two coordinates move independently, which cuts the parameter count from roughly 140k to 2.8k. We measure how much that independence assumption costs us in Section V.

Trading costs enter through the execution model, which we treat as part of the problem specification. A *taker* crosses the spread (the gap between the bid and the ask), so a LONG entry pays the ask plus the taker fee and an early exit receives the bid minus the fee. A *maker* instead rests a limit order, an order posted at a fixed price that waits passively until the market reaches it, so a LONG entry pays the bid plus the maker fee and an early exit receives the ask minus the fee. Fees follow Kalshi's official schedule, rounded up to the cent per order, with settlement fee-free [9]. With rate r ($r = 0.07$ for a taker, $r = 0.0175$ for a maker), the fee at price P is $\lceil 100 r P (1 - P) \rceil$ cents. Kalshi applies maker fees only to designated markets, and no publicly accessible primary source states KXBTC15M's classification (the official schedule defers to a rendered market list, and the public Trade API exposes no fee-category field, as we found when we queried it on June 12, 2026). We therefore charge the maker fee throughout, which is the conservative choice when evaluating maker strategies. Because the two operators define different reward kernels on top of identical dynamics, any difference in optimal value can be attributed to execution assumptions alone. The maker operator also needs a *fill model*, which we defer to Section IV.

The agent maximizes expected episode profit and loss (P&L), and since the horizon is short we use no discounting. Before solving the MDP, it is worth asking what the answer should be if the market is efficient. Let $p_t \in [0, 1]$ be the mid-price with $p_T = \mathbf{1}\{\text{YES}\}$. On an efficient market (p_t) is a martingale, so for any entry and exit times $\tau \leq \tau' \leq T$ chosen by any policy, optional stopping [10] gives

$$\mathbb{E}[\text{P\&L}] = \underbrace{\mathbb{E}[p_{\tau'} - p_{\tau}]}_{=0} - \text{costs} < 0 \quad (2)$$

under taker execution, where costs (spread plus fees) are strictly positive. Equation (2) is the null hypothesis of this project. If the market is efficient, the optimal value satisfies $V^*(t, x) \leq 0$ in every state under taker execution. So whenever our DP reports $V^* > 0$, that value must come from one of two sources, either genuine market inefficiency or error in the model we gave the DP. Distinguishing the two is the task of Section V.

IV. PROPOSED METHOD

A. Offline Component

The offline stage estimates the MDP from data and then solves it exactly. The marginals in (1) are empirical transition counts with Laplace smoothing $\alpha = 0.05$. We keep α small so that a transition never seen in training (for example, a price jump from bin 0 to bin 9 in a single minute) still receives probability below 0.1%. The outcome probability is estimated per state, with shrinkage toward the market price,

$$\hat{P}_{\text{YES}}(b, c) = \frac{\sum_i \mathbf{1}\{y_i = 1\} + w m_b}{n_{bc} + w}, \quad w = 20, \quad (3)$$

where the sum runs over training visits to (b, c) (we exclude the final candle, whose price already embeds the outcome), m_b is the bin- b mid-price, and n_{bc} is the visit count (mean ≈ 620 per cell, so the prior weight is small). As a control, the *efficiency baseline* replaces (3) with $\hat{P}_{\text{YES}} = m_b$, the market's own forecast. Under the null (2), the taker DP on this baseline should find no positive value anywhere.

We solve the finite-horizon MDP exactly by backward induction [11]. Let e_e^L, e_e^S denote entry costs and $x^L(b, e), x^S(b, e)$ early-exit values (entry cost included) under the chosen execution operator. Position values satisfy, for LONG (SHORT is symmetric),

$$\begin{aligned} V_L(T-1, b, c, e) &= \hat{P}_{\text{YES}}(b, c) - e_e^L \\ V_L(t, b, c, e) &= \max \left\{ x^L(b, e), \right. \\ &\quad \left. \mathbb{E}_{\hat{P}}[V_L(t+1, b', c', e)] \right\} \end{aligned} \quad (4)$$

and the flat-state value is

$$\begin{aligned} V(t, b, c) &= \max \left\{ \mathbb{E}_{\hat{P}}[V(t+1)], \right. \\ &\quad \mathbb{E}_{\hat{P}}[V_L(t+1, \cdot, b)], \\ &\quad \left. \mathbb{E}_{\hat{P}}[V_S(t+1, \cdot, b)] \right\} \end{aligned} \quad (5)$$

for $t_{\min} \leq t \leq T - 2$ (only the first term otherwise), with $V(T-1, \cdot) = 0$ because no order can execute on the resolution candle. Solving (4)-(5) over the 10×10 grid with entry-bin memory takes $O(TN_b^2N_c^2 + TN_b^2N_cN_e)$ operations, well under one second. The outputs are the value tables and the greedy policy tables π^* that we use online.

We validate the solver against ten instances that we can solve analytically. On martingale transitions the taker DP returns $V^* = 0$ exactly, and certain-win constructions return the values we compute by hand. This harness is also what caught the bug mentioned in the introduction, an earlier implementation of (5) that double-counted the entry cost. The corrected results in Section V supersede our Week 6-7 report, and we list the specific discrepancies there. The same harness also exposes the central execution-modeling pitfall. Under the certain-fill maker assumption, the DP on a feeless martingale returns a V^* equal to the full bid-ask spread in every interior cell, since buying at $p_t - s/2$ and exiting at $p_{\tau'} + s/2$ nets s in expectation by optional stopping, regardless of the agent's signal. Any backtest value obtained under that assumption therefore reflects the fill

model, not the market, and this is what motivates the fill-model sandwich below.

The same offline stage also trains a model-free baseline, because the tabular DP needs an estimated transition kernel and the trouble we diagnose in Section V lies in that kernel. A natural question is whether a model-free learner, which never estimates a kernel at all, does any better. This baseline is a Double DQN [12] that works directly on the full continuous state $s_t \in \mathbb{R}^{28}$ (Bitcoin and Kalshi order-book features, the time within the episode, and the agent’s inventory and unrealized P&L), with no discretization.

Q-learning [5] learns the optimal action-value function $Q^*(s, a)$, the expected return of taking action a in state s and then acting optimally. We represent it with a neural network Q_θ and regress it toward the one-step bootstrap target $r + \gamma \max_{a'} Q_\theta(s', a')$. The trouble is the max, because taking the maximum over noisy value estimates is biased upward, so the very estimation error we are trying to learn away gets baked into the target and inflates the values. Double DQN removes the bias by splitting the two jobs the max does at once. An *online* network Q_θ picks the greedy next action, and a separate *target* network $Q_{\theta-}$ scores it,

$$y = r + \gamma Q_{\theta-}(s', \arg \max_{a'} Q_\theta(s', a')), \quad (6)$$

and we take a gradient step on the squared error $(Q_\theta(s, a) - y)^2$. Because the network that selects the action and the one that scores it do not share the same estimation error, the upward bias largely cancels.

The target network $Q_{\theta-}$ is a frozen copy of Q_θ , refreshed every N steps. Freezing it is what keeps training stable, because if the target moved on every gradient step, the network would be chasing a goalpost that shifts as it learns, and the updates tend to oscillate or diverge. Holding the target fixed between refreshes turns each batch into an ordinary regression problem. We also use *experience replay*, in which every transition (s, a, r, s') is stored in a buffer, and updates draw mini-batches from it uniformly at random. Consecutive minutes of a single market are highly correlated, and training on them in order would bias the gradients toward whatever the agent happens to be doing, whereas sampling from the buffer breaks that correlation and lets each transition be reused many times.

We train in two phases because the reward is sparse. The reward is the net P&L, and most of it arrives only at settlement, so an agent that starts from random weights and explores at random spends a long time filling the buffer with uninformative trajectories before it ever sees a payoff. *Phase 1* side-steps this with behavioral cloning, pretraining Q_θ to imitate a supervised logistic-regression policy, so the agent begins from a sensible policy and a buffer of reasonable trajectories rather than noise. *Phase 2* then fine-tunes with the Double DQN update (6), adding potential-based reward shaping [7], a shaping term $F(s, s') = \gamma \Phi(s') - \Phi(s)$ that densifies the per-step learning signal while provably leaving the optimal policy unchanged.

B. Online Component

At each minute the online system observes the quotes, maps them to (b, c) , and reads off the action from $\pi^*(t, b, c)$ (or from the in-position table, indexed by the remembered entry bin). The only nontrivial part online is order management, which is where the fill model comes in. We use it in one of two modes.

Under *optimistic fills*, orders execute immediately at their own price level at the decision minute. This is the standard backtesting assumption, and it serves as an upper bound on maker performance.

Under *pessimistic fills*, a resting order executes only when the market later *trades through* its level (for a resting YES bid, the ask has to fall to or below it). Orders rest unchanged until they fill. An entry order still unfilled at the end of the episode is cancelled, and an unfilled exit order settles at the outcome. Because fills like this happen mostly when the price is already moving against the order, this mode isolates adverse selection and gives a lower bound on maker performance.

Both modes enforce the same hygiene rules, namely no entries or exits on the resolution candle $t = T - 1$ and no entries when the order book is one-sided (bid at \$0 or ask at \$1). The same lookup-and-order-management loop also runs against our live Kalshi order infrastructure (authentication, REST order placement, cancellation), which we include as deployment infrastructure, not as a source of results.

V. EXPERIMENT RESULTS

Everything in this section tests the efficient-market null (2), and if it holds, the taker DP on the efficiency baseline should find $V^* \leq 0$ everywhere. So any positive value the empirical DP reports is a model artifact, and we expect it to disappear once we move to held-out data.

A. Modeling & Simulation

Our data consist of 6,331 episodes spanning 2026-03-12 to 2026-05-21. We split them chronologically 70/10/20 into a training set of 4,431 episodes, a validation set of 633, and a test set of 1,267. All model estimation uses the training split alone. Fees follow the Kalshi schedule of Section III, and the resolution-candle and one-sided-book hygiene rules apply throughout. We report two-sided p -values, and for the headline result we also run a by-day block bootstrap, since the test span covers 15 trading days.

B. Baseline Methods

We compare the DP against four kinds of baseline, starting with two rule baselines, always-long and always-short, that take a taker entry at $t = 3$ and hold to settlement. Four supervised classifiers (logistic regression, random forest, gradient boosting, and an MLP) trade the predicted direction from Bitcoin-only features, entering as optimistic makers held to settlement. The learned-agent baseline is the Double DQN of Section IV, carried over from our earlier milestone work and evaluated here as a maker. The efficiency-baseline DP serves as the theoretical control. In Table I, the rule and DP rows each state their execution model. The four supervised rows are descriptive

TABLE I

HELD-OUT TEST RESULTS ON THE $n = 1,267$ TEST EPISODES, WITH MEAN P&L IN CENTS PER EPISODE AND TWO-SIDED p -VALUES.

Method	mean	win	t	p
Always-long (taker)	-4.1	48.9%	-3.24	0.001
Always-short (taker)	-0.7	51.1%	-0.59	0.556
LR, BTC-only	-1.4	61.5%	-1.10	0.270
Random forest	-1.2	60.9%	-0.93	0.354
Gradient boosting	+0.4	62.0%	+0.35	0.728
MLP	-0.9	61.3%	-0.67	0.500
DDQN (maker)	collapses to abstention			
DP, optimistic fills	-2.2	25.9%	-2.57	0.010
DP, pessimistic fills	-8.8	11.6%	-10.09	$< 10^{-3}$

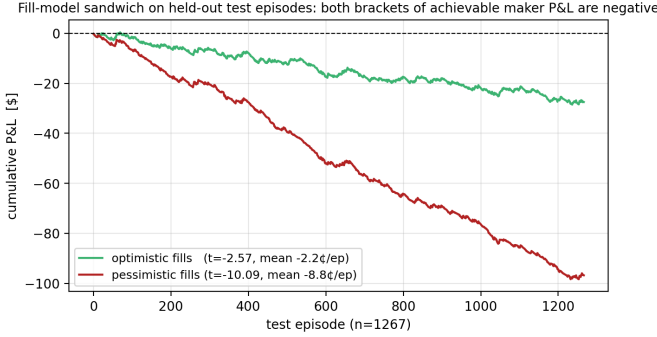


Fig. 1. Cumulative test P&L of the empirical DP policy under the optimistic (always filled) and pessimistic (trade-through only) fill models.

comparisons, and from them validation picked the MLP as the single confirmatory test.

C. Results and Discussion

The mean V^* at its chosen entries is +\$0.054 per contract, and the policy finds an entry it likes in nearly every episode. Out of sample, under optimistic fills, the policy trades 1,253 of the 1,267 test episodes (753 long, 500 short) and nets -2.2 cents per episode ($t = -2.57$, $p = 0.010$, with block-bootstrap $p < 0.001$) at a win rate of 25.9% (Table I, Fig. 1).

Fig. 1 overlays the same DP policy under both fill models. Optimistic fills net -2.2 cents per episode, while pessimistic fills net -8.8 cents ($t = -10.09$, win rate 11.6%). Real maker performance has to lie somewhere between the two, so the entire achievable band is negative. The gap between the two brackets (≈ 6.6 cents per episode) is a direct measure of how much of a backtest’s apparent quality can rest on the fill assumption alone. The pessimistic bound also isolates adverse selection (orders that fill only on trade-through are filled precisely when the market is moving against them) and the win rate falls from 25.9% to 11.6%.

Where does the in-sample value come from? Fig. 4 shows V^* at $t = 3$ for the efficiency baseline ($\hat{P}_{\text{YES}} = m_b$) under both execution operators and both transition kernels. On exact martingale dynamics (top row) the result matches (2), namely the taker value being zero everywhere, and the certain-fill

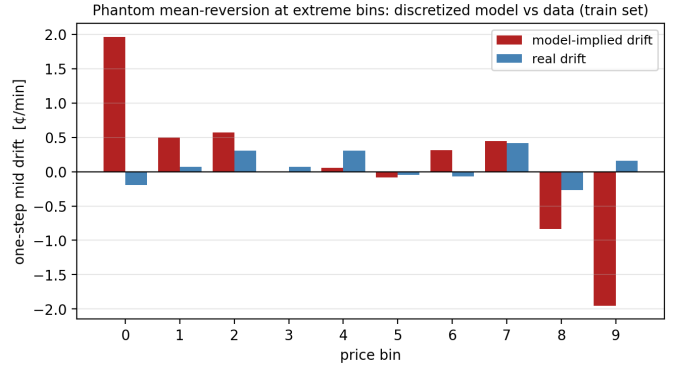


Fig. 2. Model-implied versus real one-step drift of the mid-price per price bin (train set).

maker value equalling the spread. On the *estimated* kernel (bottom row), though, even the *taker* DP claims value, up to +10.2 cents in 1,050/1,200 cells. By (2) no such value can exist, and we have already verified the solver to be exact on martingale inputs. The claimed value is therefore a certified lower bound on the estimated kernel’s value error, and we can read it off without ever knowing the true dynamics. We suggest this no-arbitrage check as a generally useful model diagnostic, since any estimated financial MDP can be audited by pricing a strategy that provably has nonpositive value.

Fig. 2 identifies the mechanism, representing bin-0 and bin-9 prices by their bin midpoints (5 and 95 cents), but true prices inside those bins actually sit nearer 3 and 97 cents. That small gap is enough to make every observed transition out of an extreme bin look like mean reversion worth about ± 2 cents per minute. The real per-minute drift of the mid-price, on the other hand, is statistically indistinguishable from zero in every bin, as (2) predicts. Handed this kernel, the DP rationally buys extreme-priced contracts to harvest a reversion that exists only in the discretization. Similarly, the extreme bins are sticky (self-transition probability ≈ 0.94 , versus ≈ 0.29 for central bins), so the factored model’s implied recovery from extreme states overstates what really happens.

The remaining question is whether the model error we measured is actually large enough to explain the whole in-sample-to-out-of-sample gap. For a policy π evaluated under the estimated kernel $\hat{P}$ instead of the true P , the standard telescoping argument [8] bounds the value error by the model’s one-step total-variation (TV) error, accumulated along the policy’s own visitation:

$$|\hat{V}^\pi - V^\pi| \leq V_{\max} \sum_u \mathbb{E}_\pi \left[\text{TV}(P(\cdot | x_u, u), \hat{P}(\cdot | x_u, u)) \right]. \quad (7)$$

We estimate the TV terms by comparing the factored kernel (1) against the raw empirical joint transition frequencies, and then subtracting a parametric-bootstrap estimate of pure sampling noise. We aggregate over time and momentum bins on cells with $n \geq 20$ and weight by visit count (Fig. 3). The excess error along the learned policy’s visitation is ≈ 0.041 per step

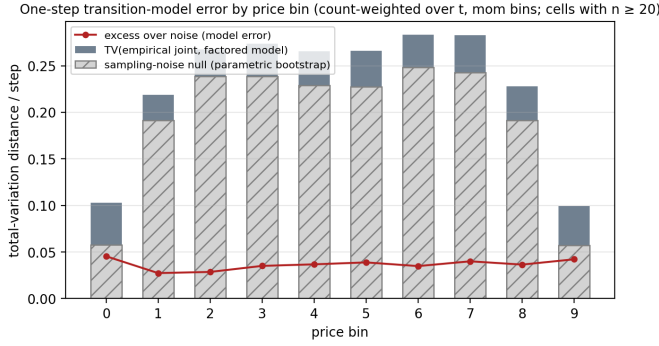


Fig. 3. One-step transition-model error by price bin. The plotted excess, the total TV against the factored model minus the parametric-bootstrap sampling-noise null, feeds the bound (7).

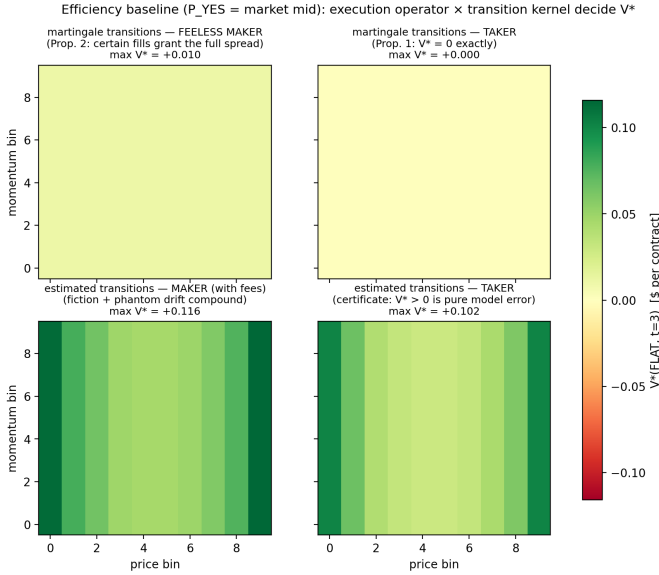


Fig. 4. $V^*(\text{FLAT}, t=3)$ under $\{\text{martingale, estimated}\}$ transitions $\times$ $\{\text{maker, taker}\}$ execution, on a shared color scale.

over 6.2 exposed steps per episode, so (7) allows value error up to $\approx \$0.25$ per episode. That is roughly five times the claimed in-sample edge of $\$0.054$ per contract. Transition-model error is therefore, on its own, large enough to account for the entire out-of-sample collapse. We need not appeal to market inefficiency, and the data do not support doing so.

The verification harness (Section IV) exposed an entry-cost double-counting error in the recursion (5) as we had previously implemented it. Entries before the final step were charged twice, which pushed the policy to defer almost all of its activity to extreme-priced, late-episode states. Table II lists the changes. Beyond the recursion fix, p -values are now two-sided rather than one-sided, entries and exits are banned on the resolution candle, the final candle is excluded from (3), and one-sided books are skipped. As Fig. 4 shows that premium is really the option value created by the certain-fill assumption, and it shows up at nearly all t and states, not just at the early steps.

TABLE II
WEEK 6-7 MILESTONE VS. CORRECTED RESULTS (HELD-OUT TEST).

	Week 6-7	Corrected
Backtest t	-3.16	-2.57
p -value	0.001 (one-sided)	0.010 (two-sided)
Win rate	4.4%	25.9%
Active episodes	all at extreme bins	1,253 across bins
Maker-baseline $V^* > 0$	early t only	1,080/1,200 cells

VI. CONCLUSION & FUTURE WORK

We solved a real prediction-market trading MDP by exact backward induction, and because the solver is exact, we were able to explain our negative result.

At this resolution, execution modeling matters more than signal modeling. The gap between fill assumptions (≈ 6.6 cents per episode) is several times larger than any candidate signal we measured. The per-trade economics make the same point from the cost side. A taker entry on a 50-cent contract costs about 51 cents at the ask plus a 2-cent fee, so a directional bet has to win about 53% of the time just to break even. Our best supervised classifiers clear that bar on raw direction (test accuracy about 61%, Table I), and yet they still lose money, because their correct calls cluster on contracts the market already prices as favorites (likely winners, quoted well above 50 cents), so being right 61% of the time on a contract priced at 65 cents still loses.

Discretization, too, is far from neutral, since a coarse price grid turned bounded prices into phantom mean reversion worth more than the fee hurdle, and the DP found and “monetized” it in sample. The no-arbitrage certificate (price a strategy that provably has no value under the estimated model, so that any positive value is model error) is cheap to run and would have flagged the problem right away. We recommend it as a standard diagnostic for estimated financial MDPs.

The study has several important limitations. First, the test set consists of only 15 trading days from a single market. Second, trade execution is approximated using minute-bar data rather than order-book information. Finally, the fill model is designed to be conservative and may understate achievable performance.

Looking ahead, the binding constraint is data resolution rather than algorithms. Collecting tick-level order-book data would open up several directions, namely a fair-value strategy that replicates the settlement index (CF Benchmarks’ Bitcoin Real-Time Index) at sub-second latency, a volatility-mispricing study (the quote implies a volatility that can be compared against realized-volatility forecasts), a calibration study of the tails motivated by the longshot-bias literature [3], and static no-arbitrage checks across related Kalshi markets. If we relax our assumption that markets are i.i.d., a hidden Markov model, itself a dynamic-programming method, could label each market by regime and switch strategies accordingly. Each of these already has its mathematical foundations and a falsifying test worked out.

REFERENCES

- [1] R. Almgren and N. Chriss, "Optimal execution of portfolio transactions," *Journal of Risk*, vol. 3, no. 2, pp. 5–39, 2001.
- [2] J. Wolfers and E. Zitzewitz, "Prediction markets," *Journal of Economic Perspectives*, vol. 18, no. 2, pp. 107–126, 2004.
- [3] E. Snowberg and J. Wolfers, "Explaining the favorite-long shot bias: Is it risk-love or misperceptions?" *Journal of Political Economy*, vol. 118, no. 4, pp. 723–746, 2010.
- [4] H. Mohanty and B. Krishnamachari, "Do prediction markets forecast cryptocurrency volatility? Evidence from Kalshi macro contracts," arXiv:2604.01431 [q-fin.ST], 2026.
- [5] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992.
- [6] T. G. Fischer, "Reinforcement learning in financial markets - a survey," Friedrich-Alexander University Erlangen-Nuremberg, Institute for Economics, FAU Discussion Papers in Economics 12/2018, 2018.
- [7] A. Y. Ng, D. Harada, and S. J. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in *Proceedings of the 16th International Conference on Machine Learning (ICML)*, 1999, pp. 278–287.
- [8] M. Kearns and S. Singh, "Near-optimal reinforcement learning in polynomial time," *Machine Learning*, vol. 49, no. 2–3, pp. 209–232, 2002.
- [9] Kalshi, "Fee schedule," <https://kalshi.com/docs/kalshi-fee-schedule.pdf>, 2026, effective Feb 5, 2026; retrieved June 12, 2026.
- [10] D. Williams, *Probability with Martingales*. Cambridge University Press, 1991.
- [11] D. P. Bertsekas, *Dynamic Programming and Optimal Control, Vol. I*, 4th ed. Belmont, MA: Athena Scientific, 2017.
- [12] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence (AAAI-16)*, vol. 30, no. 1, 2016, pp. 2094–2100.